```
using FluentAssertions;
using FractionLib;
using Xunit;

namespace FractionLib.Tests
{
  public class FractionTests
  {
    // ------------------------------------
---------------------------
    // Конструктор
    // ------------------------------------
---------------------------

    [Fact]
    public void
Ctor_ZeroDenominator_ThrowsDivideByZeroExce
ption()
    {
      // Arrange
      Action act = () => new Fraction(1,
0);

      // Act & Assert

act.Should().Throw<DivideByZeroException>()
         .WithMessage("Denominator cannot
be zero.");
    }
```

```
    [Fact]
    public void
Ctor_ZeroNumeratorAndZeroDenominator_Throws
DivideByZeroException()
    {
      Action act = () => new Fraction(0,
0);


act.Should().Throw<DivideByZeroException>
();
    }

    [Theory]
    [InlineData(3, 7, 3, 7)]     // уже
несократимая
    [InlineData(2, 4, 1, 2)]     //
сокращение
    [InlineData(6, 3, 2, 1)]     // целое
число
    [InlineData(5, 1, 5, 1)]     //
знаменатель 1
    [InlineData(-2, 4, -1, 2)]   //
отрицательный числитель
    [InlineData(2, -4, -1, 2)]   //
отрицательный знаменатель -> знак
переносится
    [InlineData(-2, -4, 1, 2)]   // оба
отрицательные
    [InlineData(0, 5, 0, 1)]     // ноль
    [InlineData(0, -5, 0, 1)]    // ноль с
отрицательным знаменателем
    public void
Ctor_ValidValues_NormalizesFraction(
```

```
        int numerator, int denominator, int
expectedNumerator, int expectedDenominator)
    {
      // Arrange & Act
      var fraction = new
Fraction(numerator, denominator);

      // Assert

fraction.Numerator.Should().Be(expectedNume
rator);

fraction.Denominator.Should().Be(expectedDe
nominator);
    }

    // ---------------------------------
-------------------------
    // Оператор +
    // ---------------------------------
-------------------------

    public static IEnumerable<object[]>
AddData => new List<object[]>
    {
      //           aN, aD, bN, bD, expN,
expD
      new object[] { 1, 2, 1, 3, 5, 6
},     // разные знаменатели
      new object[] { 1, 2, 1, 2, 1, 1
},     // результат - целое
      new object[] { 1, 4, 1, 4, 1, 2
},     // результат сокращается
      new object[] { -1, 2, 1, 2, 0, 1
```

```
},    // результат ноль
      new object[] { -1, 3, -1, 6, -1, 2
},  // отрицательные
      new object[] { 0, 1, 3, 4, 3, 4
},     // прибавление нуля
    };

    [Theory]
    [MemberData(nameof(AddData))]
    public void
Add_TwoFractions_ReturnsSum(
        int aN, int aD, int bN, int bD, int
expN, int expD)
    {
      // Arrange
      var a = new Fraction(aN, aD);
      var b = new Fraction(bN, bD);

      // Act
      var result = a + b;

      // Assert
      result.Numerator.Should().Be(expN);
      result.Denominator.Should().Be(expD);
    }

    // ------------------------------------
--------------------------
    // Оператор -
    // ------------------------------------
--------------------------

    public static IEnumerable<object[]>
SubtractData => new List<object[]>
```

```
    {
      new object[] { 1, 2, 1, 3, 1, 6 },
      new object[] { 1, 3, 1, 2, -1, 6
},    // отрицательный результат
      new object[] { 3, 4, 3, 4, 0, 1
},     // результат ноль
      new object[] { 1, 2, -1, 2, 1, 1
},    // вычитание отрицательной
      new object[] { 5, 6, 1, 3, 1, 2
},     // результат сокращается
    };

    [Theory]
    [MemberData(nameof(SubtractData))]
    public void
Subtract_TwoFractions_ReturnsDifference(
        int aN, int aD, int bN, int bD, int
expN, int expD)
    {
      var a = new Fraction(aN, aD);
      var b = new Fraction(bN, bD);

      var result = a - b;

      result.Numerator.Should().Be(expN);
      result.Denominator.Should().Be(expD);
    }

    // ------------------------------------
---------------------------
    // Оператор *
    // ------------------------------------
---------------------------
```

```
    public static IEnumerable<object[]>
MultiplyData => new List<object[]>
    {
      new object[] { 1, 2, 2, 3, 1, 3
},     // с сокращением
      new object[] { 2, 3, 3, 2, 1, 1
},     // результат - 1
      new object[] { -1, 2, 1, 3, -1, 6
},   // разные знаки
      new object[] { -1, 2, -1, 3, 1, 6
},   // оба отрицательные
      new object[] { 0, 1, 5, 7, 0, 1
},     // умножение на ноль
    };

    [Theory]
    [MemberData(nameof(MultiplyData))]
    public void
Multiply_TwoFractions_ReturnsProduct(
        int aN, int aD, int bN, int bD, int
expN, int expD)
    {
      var a = new Fraction(aN, aD);
      var b = new Fraction(bN, bD);

      var result = a * b;

      result.Numerator.Should().Be(expN);
      result.Denominator.Should().Be(expD);
    }

    // ------------------------------------
-------------------------
    // Оператор /
```

```
    // ------------------------------------
--------------------------

    public static IEnumerable<object[]>
DivideData => new List<object[]>
    {
      new object[] { 1, 2, 1, 4, 2, 1
},     // обычное деление
      new object[] { 1, 2, 3, 4, 2, 3 },
      new object[] { 1, 2, -1, 4, -2, 1
},   // деление на отрицательную
      new object[] { -1, 2, -1, 4, 2, 1
},   // обе отрицательные
      new object[] { 0, 1, 3, 4, 0, 1
},     // ноль делить на дробь
    };

    [Theory]
    [MemberData(nameof(DivideData))]
    public void
Divide_TwoFractions_ReturnsQuotient(
        int aN, int aD, int bN, int bD, int
expN, int expD)
    {
      var a = new Fraction(aN, aD);
      var b = new Fraction(bN, bD);

      var result = a / b;

      result.Numerator.Should().Be(expN);
      result.Denominator.Should().Be(expD);
    }

    [Fact]
```

```
    public void
Divide_ByZeroFraction_ThrowsDivideByZeroExc
eption()
    {
      var a = new Fraction(1, 2);
      var zero = new Fraction(0, 1);

      Action act = () => { Fraction _ = a /
zero; };


act.Should().Throw<DivideByZeroException>()
         .WithMessage("Cannot divide by
zero.");
    }

    // ------------------------------------
--------------------------
    // ToDouble
    // ------------------------------------
--------------------------

    [Theory]
    [InlineData(1, 2, 0.5)]
    [InlineData(-1, 4, -0.25)]
    [InlineData(0, 3, 0.0)]
    [InlineData(5, 1, 5.0)]
    [InlineData(1, 3, 0.3333333333)]
    public void
ToDouble_ReturnsDecimalValue(int n, int d,
double expected)
    {
      var fraction = new Fraction(n, d);
```

```
      var result = fraction.ToDouble();


result.Should().BeApproximately(expected,
1e-9);
    }

    // ------------------------------------
-------------------------
    // ToInt
    // ------------------------------------
-------------------------

    [Theory]
    [InlineData(6, 3, 2)]     // целое
    [InlineData(7, 2, 3)]     // усечение
вниз
    [InlineData(-7, 2, -3)]   // усечение к
нулю для отрицательных
    [InlineData(1, 2, 0)]     // меньше
единицы
    [InlineData(0, 5, 0)]
    public void
ToInt_ReturnsTruncatedValue(int n, int d,
int expected)
    {
      var fraction = new Fraction(n, d);

      var result = fraction.ToInt();

      result.Should().Be(expected);
    }

    // ------------------------------------
```

```
--------------------------
    // ToString
    // ------------------------------------
--------------------------

    [Theory]
    [InlineData(1, 2, "1/2")]
    [InlineData(-3, 4, "-3/4")]
    [InlineData(2, -4, "-1/2")]
    [InlineData(0, 5, "0/1")]
    [InlineData(4, 2, "2/1")]
    public void
ToString_ReturnsNumeratorSlashDenominator(i
nt n, int d, string expected)
    {
      var fraction = new Fraction(n, d);

      var result = fraction.ToString();

      result.Should().Be(expected);
    }

    // ------------------------------------
--------------------------
    // Equals(Fraction)
    // ------------------------------------
--------------------------

    [Fact]
    public void
EqualsFraction_SameValueDifferentRepresenta
tion_ReturnsTrue()
    {
      var a = new Fraction(2, 4);
```

```
      var b = new Fraction(1, 2);

      var result = a.Equals(b);

      result.Should().BeTrue();
    }

    [Theory]
    [InlineData(1, 2, 1, 3)]    // разные
знаменатели
    [InlineData(1, 2, 2, 3)]    // разные
числители и знаменатели
    [InlineData(1, 2, -1, 2)]   // разные
знаки
    [InlineData(2, 3, 1, 3)]    // разные
числители
    public void
EqualsFraction_DifferentValues_ReturnsFalse
(int aN, int aD, int bN, int bD)
    {
      var a = new Fraction(aN, aD);
      var b = new Fraction(bN, bD);

      var result = a.Equals(b);

      result.Should().BeFalse();
    }

    // ------------------------------------
---------------------------
    // Equals(object)
    // ------------------------------------
---------------------------
```

```
    [Fact]
    public void
EqualsObject_BoxedEqualFraction_ReturnsTrue
()
    {
      var a = new Fraction(1, 2);
      object b = new Fraction(2, 4);

      var result = a.Equals(b);

      result.Should().BeTrue();
    }

    [Fact]
    public void
EqualsObject_BoxedDifferentFraction_Returns
False()
    {
      var a = new Fraction(1, 2);
      object b = new Fraction(1, 3);

      var result = a.Equals(b);

      result.Should().BeFalse();
    }

    [Fact]
    public void
EqualsObject_Null_ReturnsFalse()
    {
      var a = new Fraction(1, 2);

      var result = a.Equals(null);
```

```
      result.Should().BeFalse();
    }

    [Theory]
    [InlineData("1/2")]
    [InlineData(1)]
    [InlineData(0.5)]
    public void
EqualsObject_OtherType_ReturnsFalse(object
other)
    {
      var a = new Fraction(1, 2);

      var result = a.Equals(other);

      result.Should().BeFalse();
    }

    // ------------------------------------
---------------------------
    // GetHashCode
    // ------------------------------------
---------------------------

    [Fact]
    public void
GetHashCode_EqualFractions_ReturnSameHash()
    {
      var a = new Fraction(1, 2);
      var b = new Fraction(-3, -6);

      var hashA = a.GetHashCode();
      var hashB = b.GetHashCode();
```

```
      hashA.Should().Be(hashB);
    }

    [Fact]
    public void
GetHashCode_DifferentFractions_ReturnDiffer
entHash()
    {
      var a = new Fraction(1, 2);
      var b = new Fraction(1, 3);

      var hashA = a.GetHashCode();
      var hashB = b.GetHashCode();

      hashA.Should().NotBe(hashB);
    }

    // ------------------------------------
---------------------------
    // CompareTo
    // ------------------------------------
---------------------------

    [Theory]
    [InlineData(1, 2, 1, 3)]     // больше
    [InlineData(1, 2, -1, 2)]    //
положительная > отрицательной
    [InlineData(-1, 3, -1, 2)]   // среди
отрицательных
    public void
CompareTo_GreaterFraction_ReturnsPositive(i
nt aN, int aD, int bN, int bD)
    {
      var a = new Fraction(aN, aD);
```

```
      var b = new Fraction(bN, bD);

      var result = a.CompareTo(b);

      result.Should().BePositive();
    }

    [Theory]
    [InlineData(1, 3, 1, 2)]     // меньше
    [InlineData(-1, 2, 1, 2)]    //
отрицательная < положительной
    [InlineData(-1, 2, -1, 3)]   // среди
отрицательных
    public void
CompareTo_LesserFraction_ReturnsNegative(in
t aN, int aD, int bN, int bD)
    {
      var a = new Fraction(aN, aD);
      var b = new Fraction(bN, bD);

      var result = a.CompareTo(b);

      result.Should().BeNegative();
    }

    [Fact]
    public void
CompareTo_EqualFraction_ReturnsZero()
    {
      var a = new Fraction(1, 2);
      var b = new Fraction(2, 4);

      var result = a.CompareTo(b);
```

```
      result.Should().Be(0);
    }

    [Fact]
    public void
CompareTo_UsedForSorting_OrdersFractionsAsc
ending()
    {
      var list = new List<Fraction>
      {
        new(1, 2), new(-1, 3), new(3, 4),
new(0, 1)
      };

      list.Sort();

      list.Should().Equal(new Fraction(-1,
3), new Fraction(0, 1),
                          new Fraction(1,
2), new Fraction(3, 4));
    }
  }
}
```